RETAILMART V3 · SQL

SQL Subqueries Part 1 — Scalar, IN, Derived Tables

— Siraj Sir

📱 WhatsApp Announcement

SUBQUERIES PART 1

Until now, every WHERE clause needed a hardcoded number — 'salary > 50000', 'orders > 100'. Today that ends. Subqueries make those numbers DYNAMIC. 'WHERE salary > (the average)' where the average is computed live by another query. This is where SQL grows up.

Today's Focus:

- Scalar subqueries — return ONE value, use in WHERE or SELECT
- IN-subqueries — filter against a dynamic list of values
- Derived tables — subqueries in FROM (pre-aggregation)
- Rewriting IN-subqueries as JOINs

Tomorrow we add correlated subqueries, EXISTS, and CTEs — but today's three patterns alone cover 80% of analyst needs.
— Siraj Sir
#Day12 #PostgreSQL #Subqueries #DerivedTables

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	SELF JOIN + UNION recap, When to use each
Part 1: Scalar Subqueries	40 mins	Subquery returning ONE value, In WHERE for dynamic comparisons, In SELECT for computed benchmark columns, 4 practice problems (Easy → Crazy)
Part 2: IN-Subqueries	40 mins	IN — filter by dynamic list membership, NOT IN — the NULL trap, Rewriting IN-subqueries as JOINs, 4 practice problems (Easy → Crazy)
Part 3: Derived Tables (FROM)	40 mins	Subquery as a table in FROM clause, Pre-aggregation pattern (prevents fan-out), Joining multiple derived tables, 4 practice problems (Easy → Crazy)
Quiz + Summary & Wrap-up	15 mins	Quick quiz, Common mistakes, Homework, CTEs preview

YOUR SQL JOURNEY

- ✓ CASE & COALESCE
- ✓ Aggregates & GROUP BY
- ✓ JOINs Part 1
- ✓ JOINs Part 2
- ✓ Self Join & UNION
- 12 **Subqueries Part 1** ← HERE
- 13 Subqueries Part 2 + CTEs
- 14 DB Concepts

RECAP

Memory Check

SELF JOIN (same table, two aliases, row-by-row comparison) and UNION/UNION ALL (stacking results vertically). That wraps the combine-data toolkit.

Today: A new dimension — putting queries INSIDE other queries. Static values become DYNAMIC computations. The query body becomes nestable, like functions in code. This is the foundation of complex analytics.

STORY

The Above-Average Student Question

A teacher asks: 'Which students scored above the class average?' To answer, she FIRST calculates the average (one number), then compares each student's score to it. That single number — the average — is a SUBQUERY. One small query feeds a value into another query.

Real analyst questions almost always look like this: 'Find X where some computed property of X matches some computed benchmark.' Both 'X' and 'benchmark' can themselves be queries.

REVEAL

From Story to SQL!

'Above average' → WHERE col > (SELECT AVG(...) FROM ...)

'In a list' → WHERE col IN (SELECT col FROM ...)

'Pre-aggregate then filter' → FROM (SELECT ... GROUP BY ...) AS t

'Show alongside benchmark' → scalar subquery in SELECT

All three are subqueries — same idea, different positions

DEFINITION

The Subquery Family (Part 1)

1. Scalar Subquery — returns ONE value (1 row, 1 column)
Use: in SELECT as a column, in WHERE for comparison

2. Multi-row Subquery — returns MANY rows (1 column)
Use: with IN, NOT IN

3. Derived Table — returns a TABLE (many rows, many columns)
Use: in FROM clause as if it were a real table

Tomorrow's Part 2: correlated subqueries + EXISTS + CTEs

Anatomy of a Subquery

```
-- Inner query runs FIRST, outer query uses result
SELECT first_name, last_name, salary
FROM stores.employees
WHERE salary > (
  SELECT AVG(salary) FROM stores.employees
); -- ← This is the SUBQUERY

-- Step 1: Inner query returns one number (avg)
-- Step 2: Outer query uses it in comparison
-- The DB optimizes execution but logically inside-out
```

STORY

The Class Average Comparison

To find students above the class average, you FIRST calculate the average (one number), then compare each student to it. That single number is a SCALAR SUBQUERY — one query, one value, used inside another query.

DEFINITION

Scalar Subquery

Technical:

A scalar subquery returns EXACTLY ONE value — one row, one column. It can appear anywhere a literal value can: in SELECT (as a column), WHERE (for comparison), or HAVING. If the subquery returns multiple rows, you get an error.

Layman's:

Scalar subquery = 'Compute one number/value here, then use it as if it were typed in.' Replace hardcoded values with dynamic ones. WHERE salary > 50000 becomes WHERE salary > (compute the average dynamically).

SYNTAX: Scalar Subqueries (1/2)

```
-- In WHERE (for comparison)
SELECT first_name, salary FROM stores.employees
WHERE salary > (
  SELECT AVG(salary) FROM stores.employees
);

-- In SELECT (as a computed column)
SELECT first_name, salary,
  (SELECT AVG(salary) FROM stores.employees)
    AS company_avg,
  salary - (SELECT AVG(salary)
    FROM stores.employees)
    AS diff_from_avg
FROM stores.employees;

-- In HAVING (filter aggregated groups)
SELECT dept_id, COUNT(*)
FROM stores.employees GROUP BY dept_id
HAVING COUNT(*) > (
  SELECT COUNT(*) * 1.0 / COUNT(DISTINCT dept_id)
```

SYNTAX: Scalar Subqueries (2/2)

```
  FROM stores.employees
);
```

Step-by-Step: Scalar Subquery Execution

```
-- Query:
-- SELECT name, salary FROM employees
-- WHERE salary > (SELECT AVG(salary)
--                 FROM employees);

-- Step 1: Run inner query
-- SELECT AVG(salary) FROM employees → 65000

-- Step 2: Substitute the value
-- SELECT name, salary FROM employees
-- WHERE salary > 65000;

-- Step 3: Execute outer query
-- Returns all employees earning more than 65000
```

Easy: Customers Who Spent Above the Global Average

EASY

SCENARIO: The CMO wants high-spend customers — those whose single largest order exceeded the COMPANY-WIDE average order value. These are VIP outreach candidates.

YOUR TASK: Compute the average net_total across all orders (one number). Filter orders where net_total exceeds that. Show customer name, order_id, net_total.

🤔 **Hint:** The 'average' is one number computed over the whole table — perfect for a scalar subquery in WHERE. Trap: a scalar subquery MUST return exactly one row; SELECT net_total (without an aggregate) would return many rows and fail with 'more than one row returned'.

Answer

✅ ANSWER

```
SELECT
  c.first_name || ' ' || c.last_name
    AS customer,
  o.order_id,
  o.net_total
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
WHERE o.net_total > (
  SELECT AVG(net_total)
  FROM sales.orders
)
ORDER BY o.net_total DESC
LIMIT 15;
```

Medium: Products Cheaper Than Their Category Average

MEDIUM

SCENARIO: The pricing team wants to find products priced BELOW the global average — these are entry-level products. Show product, price, the global average, and the gap.

YOUR TASK: In SELECT, include a scalar subquery for the average product price as a benchmark column. In WHERE, filter products with price below that average. Show product_name, price, avg_price, gap.

🤔 **Hint:** A scalar subquery can sit RIGHT in the SELECT list as a computed column — every row gets the same benchmark. Trap: if you reference the alias in another expression on the same SELECT level, PostgreSQL won't recognize it — repeat the full subquery, or wrap the query in an outer one.

Answer

✅ ANSWER

```
SELECT
  product_name,
  price,
  (SELECT ROUND(AVG(price), 0)
   FROM products.products)
    AS avg_price,
  ROUND((SELECT AVG(price)
    FROM products.products), 0)
    - price AS price_gap
FROM products.products
WHERE price < (
  SELECT AVG(price)
  FROM products.products
)
ORDER BY price ASC LIMIT 20;
```

Hard: HR Salary History — Latest vs Initial Salary

HARD

SCENARIO: The CHRO wants a salary growth report: per employee, show their CURRENT salary alongside their FIRST salary on record. Use scalar subqueries to fetch both values for each employee. Both come from hr.salary_history.

YOUR TASK: For each employee, include scalar subqueries returning their latest and earliest payment amounts from hr.salary_history. Show name, latest_salary, first_salary, growth amount, and growth %.

🤔 **Hint:** Two scalar subqueries can sit side-by-side in SELECT — one for MAX(payment_date) salary, one for MIN(payment_date) salary. Trap: subqueries that return ORDER BY … LIMIT 1 only return one row, so they're scalar-safe — but reading them in pairs gets verbose; use MAX/MIN aggregates wrapped in a CTE if performance matters.

Answer (1/2)

✅ ANSWER

```
SELECT
  e.employee_id,
  e.first_name || ' ' || e.last_name
    AS employee,
  e.role,
  (SELECT amount FROM hr.salary_history sh
   WHERE sh.employee_id = e.employee_id
   ORDER BY payment_date DESC LIMIT 1)
    AS latest_salary,
  (SELECT amount FROM hr.salary_history sh
   WHERE sh.employee_id = e.employee_id
   ORDER BY payment_date ASC LIMIT 1)
    AS first_salary,
  (SELECT amount FROM hr.salary_history sh
   WHERE sh.employee_id = e.employee_id
   ORDER BY payment_date DESC LIMIT 1)
  - (SELECT amount FROM hr.salary_history sh
     WHERE sh.employee_id = e.employee_id
     ORDER BY payment_date ASC LIMIT 1)
    AS growth_amount
FROM stores.employees e
WHERE EXISTS (
```

Answer (2/2)

✅ ANSWER

```
  SELECT 1 FROM hr.salary_history sh
  WHERE sh.employee_id = e.employee_id
)
ORDER BY growth_amount DESC
  NULLS LAST LIMIT 15;
```

Crazy: Multi-Benchmark Performance Card

CRAZY

SCENARIO: The CFO wants a product performance card showing total revenue alongside FOUR benchmark columns: company avg revenue, top 10% threshold, max revenue, and median. Four scalar subqueries in one SELECT.

YOUR TASK: Pre-aggregate revenue per product in a derived table. Then in SELECT, reference scalar subqueries for each benchmark. Use PERCENTILE_CONT for median and P90. Show top 10 products by revenue.

🤔 **Hint:** Each benchmark is its own scalar subquery against the same product-revenue set — compute the set ONCE, then reuse it for all four benchmarks. Trap: repeating the inner aggregation four times in raw SQL is slow and error-prone — a derived table you query four ways is cleaner.

Answer (1/2)

✅ ANSWER

```
SELECT
  pr.product_name,
  pr.revenue,
  (SELECT ROUND(AVG(revenue)::NUMERIC, 0)
   FROM (SELECT prod_id,
     SUM(net_amount) AS revenue
     FROM sales.order_items
     GROUP BY prod_id) x)
    AS avg_rev,
  (SELECT ROUND(
    (PERCENTILE_CONT(0.9)
    WITHIN GROUP (ORDER BY revenue))::NUMERIC, 0)
   FROM (SELECT prod_id,
     SUM(net_amount) AS revenue
     FROM sales.order_items
     GROUP BY prod_id) x)
    AS top_10_pct,
  (SELECT MAX(revenue)
   FROM (SELECT prod_id,
     SUM(net_amount) AS revenue
     FROM sales.order_items
     GROUP BY prod_id) x)
```

Answer (2/2)

✅ ANSWER

```
    AS max_rev
FROM (
  SELECT p.product_id,
    p.product_name,
    ROUND(SUM(oi.net_amount), 0)
      AS revenue
  FROM products.products p
  JOIN sales.order_items oi
    ON p.product_id = oi.prod_id
  GROUP BY p.product_id, p.product_name
) pr
ORDER BY pr.revenue DESC LIMIT 10;
```

KEY TAKEAWAYS

Scalar Subqueries

Scalar subquery = inner query returning ONE value. Use in WHERE for dynamic comparison, in SELECT as computed column, in HAVING for group-level filtering. Multiple scalar subqueries can appear in one SELECT. ERROR if subquery returns more than one row — aggregate it (MAX/MIN/AVG) or use IN instead.

COMMON MISTAKE

Scalar Subquery Returning Multiple Rows

The Mistake:
WHERE salary > (SELECT salary FROM employees WHERE dept_id = 5) — if dept 5 has multiple employees, the subquery returns many rows. PostgreSQL throws: 'more than one row returned by a subquery used as an expression'.

The Fix:
Either aggregate to one row: SELECT MAX(salary) or AVG(salary). Or use IN instead of > for multi-row subqueries: WHERE salary IN (SELECT salary FROM ...). Choose based on intent.

STORY

The VIP Guest List

A wedding planner has a VIP guest list (many names). Each table at the venue needs to know: 'Is THIS guest on the VIP list?' That is exactly the IN operator. Check membership against a dynamic list. NOT IN = the reverse: 'is this guest NOT on the VIP list?'

DEFINITION

IN and NOT IN

Technical:
IN (subquery) = TRUE if the value matches ANY row in the subquery result. NOT IN = TRUE if it matches NONE. The subquery can return MULTIPLE rows (unlike scalar subqueries). Equivalent to chaining many OR conditions, but dynamic.

Layman's:
IN = 'is this value in this list?' NOT IN = 'is this value NOT in this list?' (watch the NULL trap!). The list is computed by another query, so it can be any size.

SYNTAX: IN, NOT IN

```sql
-- IN with subquery
SELECT * FROM products.products p
WHERE p.product_id IN (
  SELECT DISTINCT prod_id
  FROM sales.order_items
);

-- NOT IN (warning: NULLs!)
SELECT * FROM customers.customers
WHERE customer_id NOT IN (
  SELECT cust_id FROM sales.orders
  WHERE cust_id IS NOT NULL  -- safety!
);

-- IN with a static list (no subquery)
SELECT * FROM sales.orders
WHERE order_status IN ('Delivered', 'Shipped');

-- IN-subquery vs JOIN — same result, different shape
-- (we'll rewrite this pattern in a practice problem)
```

Easy: Products That Have Been Ordered

EASY

SCENARIO: The merchandiser wants the list of products that have AT LEAST one order. The easy way: filter products WHERE product_id IN the list of ordered prod_ids.

YOUR TASK: Outer query: SELECT products. Inner query: distinct prod_ids from order_items. IN filters products to those appearing in the inner list. Show product_id, name, price.

🤔 **Hint:** 'Has been ordered' = product appears at least once in the order_items table. Which operator tests membership against a dynamic list? Trap: the inner list can have duplicates (one product = many order rows) — that's harmless for IN, but if you were to use NOT IN, you'd need DISTINCT and a NULL filter.

Answer

✅ ANSWER

```
SELECT
  p.product_id,
  p.product_name,
  p.price
FROM products.products p
WHERE p.product_id IN (
  SELECT DISTINCT prod_id
  FROM sales.order_items
)
ORDER BY p.price DESC
LIMIT 20;
```

Medium: Rewrite IN-Subquery as a JOIN

MEDIUM

SCENARIO: A senior engineer pushed back on your IN-subquery in a code review: 'rewrite this as a JOIN, it's clearer'. Same business question, different SQL shape — two ways to express 'customers who ordered Electronics'.

YOUR TASK: First write it as an IN-subquery (inner: customer IDs who bought Electronics). Then REWRITE the same logic as a multi-table JOIN with DISTINCT. Show both queries — they should return the same rows.

🤔 **Hint:** IN-subquery and JOIN+DISTINCT often produce identical results — choose based on readability. Trap: a JOIN without DISTINCT will duplicate customers who bought multiple Electronics — the optimizer may run different plans, but always verify with COUNT.

Answer (1/2)

✅ ANSWER

```
-- Version 1: IN-subquery
SELECT c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer
FROM customers.customers c
WHERE c.customer_id IN (
  SELECT DISTINCT o.cust_id
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
  JOIN core.dim_category cat
    ON b.category_id = cat.category_id
  WHERE cat.category_name = 'Electronics'
) LIMIT 15;

-- Version 2: JOIN with DISTINCT
SELECT DISTINCT c.customer_id,
  c.first_name || ' ' || c.last_name
```

Answer (2/2)

✅ ANSWER

```
    AS customer
FROM customers.customers c
JOIN sales.orders o
  ON c.customer_id = o.cust_id
JOIN sales.order_items oi
  ON o.order_id = oi.order_id
JOIN products.products p
  ON oi.prod_id = p.product_id
JOIN core.dim_brand b
  ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
WHERE cat.category_name = 'Electronics'
LIMIT 15;
```

Hard: Find the Bug — Multiple Rows in Scalar Position

HARD

SCENARIO: Your colleague's query is throwing 'more than one row returned by a subquery used as an expression'. They expected one number; the subquery returns many. Spot the bug and fix it two different ways.

YOUR TASK: Given a buggy query (WHERE salary > (SELECT salary FROM employees WHERE dept_id = 5)) — identify why it fails. Then show TWO fixes: (1) aggregate to one row with MAX, (2) use IN instead of >.

🤔 **Hint:** PostgreSQL accepts a scalar subquery only when it returns exactly one row. Trap: the choice between MAX and IN changes the BUSINESS MEANING — > (MAX) finds salaries higher than EVERY dept 5 employee, IN finds salaries that MATCH any dept 5 salary. Different questions!

Answer (1/2)

✅ ANSWER

```
-- Buggy original (throws error):
-- WHERE salary > (SELECT salary
--   FROM stores.employees WHERE dept_id = 5);

-- Fix 1: Aggregate to ONE row
-- (greater than every dept 5 salary)
SELECT first_name, salary
FROM stores.employees
WHERE salary > (
  SELECT MAX(salary)
  FROM stores.employees
  WHERE dept_id = 5
)
ORDER BY salary DESC LIMIT 10;

-- Fix 2: Use IN
-- (match any dept 5 salary exactly)
SELECT first_name, salary
FROM stores.employees
WHERE salary IN (
  SELECT salary
  FROM stores.employees
```

Answer (2/2)

✅ ANSWER

```
  WHERE dept_id = 5
)
ORDER BY salary DESC LIMIT 10;
```

Crazy: Cross-Sell Targets — IN + NOT IN

CRAZY

SCENARIO: The CMO wants cross-sell targets: customers who bought from 'Apparel' but NOT from 'Footwear'. Combine an inclusion filter (IN) with an exclusion filter (NOT IN). These are prime targets for footwear ads.

YOUR TASK: Two inner subqueries: one returning cust_ids who bought Apparel, one returning cust_ids who bought Footwear. Outer: WHERE IN (first) AND NOT IN (second).

🤔 **Hint:** Two filters combine: 'has bought X' (inclusion) and 'has not bought Y' (exclusion). Trap: the exclusion side has a famous NULL pitfall — if any value in the NOT IN list is NULL, the entire condition returns UNKNOWN and the query gives back zero rows. Always WHERE col IS NOT NULL in the inner query.

Answer (1/2)

✅ ANSWER

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  c.email
FROM customers.customers c
WHERE c.customer_id IN (
  SELECT DISTINCT o.cust_id
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
  JOIN core.dim_category cat
    ON b.category_id = cat.category_id
  WHERE cat.category_name = 'Apparel'
    AND o.cust_id IS NOT NULL
)
AND c.customer_id NOT IN (
  SELECT DISTINCT o.cust_id
```

Answer (2/2)

✅ ANSWER

```
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
  JOIN core.dim_category cat
    ON b.category_id = cat.category_id
  WHERE cat.category_name = 'Footwear'
    AND o.cust_id IS NOT NULL
)
LIMIT 20;
```

KEY TAKEAWAYS

IN / NOT IN

IN (subquery) — value is in the result set. NOT IN — value is NOT in the result set (watch for NULLs!). The inner query can return many rows. IN and JOIN+DISTINCT often produce the same result; choose based on readability. Tomorrow's EXISTS is the safer alternative for NOT IN.

COMMON MISTAKE

The NOT IN NULL Trap

The Mistake:
WHERE id NOT IN (SELECT col FROM table) — if the inner query returns ANY NULL, the entire NOT IN returns UNKNOWN (treated as FALSE). Your query returns ZERO rows even when matches exist.

The Fix:
ALWAYS add WHERE col IS NOT NULL in the inner query. Or use NOT EXISTS (tomorrow) — it handles NULLs correctly. NOT EXISTS is the safer pattern for exclusion queries.

STORY

The Pre-Made Sub-Report

A school principal wants to know which classes are below average attendance. Step 1: calculate average attendance PER CLASS (a sub-report). Step 2: filter classes whose attendance is below the average. That sub-report is a DERIVED TABLE — a query result used as if it were a real table.

DEFINITION

Derived Table

Technical:
A derived table is a subquery in the FROM clause, used as if it were a real table. Syntax: FROM (SELECT ...) AS alias. The inner query runs first, producing a virtual table. The outer query then SELECTs from it. Aliasing is REQUIRED.

Layman's:
Derived table = 'Run this inner query, treat its output as a table, then query the result.' Used for pre-aggregation (compute totals per group, then filter or join). Tomorrow we meet CTEs — the same idea but with cleaner syntax.

SYNTAX: Derived Tables (1/2)

```
-- Basic derived table
SELECT *
FROM (
  SELECT store_id, SUM(net_total) AS revenue
  FROM sales.orders
  GROUP BY store_id
) AS store_rev
WHERE revenue > 100000;

-- Joining derived table to a real table
SELECT s.store_name, rev.revenue
FROM stores.stores s
JOIN (
  SELECT store_id, SUM(net_total) AS revenue
  FROM sales.orders
  GROUP BY store_id
) AS rev ON s.store_id = rev.store_id
ORDER BY rev.revenue DESC;

-- Multiple derived tables in one query
```

SYNTAX: Derived Tables (2/2)

```
FROM (...) AS t1 JOIN (...) AS t2 ON ...;
```

Easy: Top Stores by Revenue (Pre-Aggregate)

EASY

SCENARIO: The CFO wants top 10 stores by total revenue. First aggregate revenue per store in a derived table, then JOIN to get store details and sort.

YOUR TASK: Derived table: SUM(net_total) GROUP BY store_id. Outer query: JOIN to stores for name and city. ORDER BY revenue DESC LIMIT 10.

🤔 **Hint:** Pre-aggregation flips the query order: GROUP BY first (inside a derived table), then JOIN to get names. Trap: PostgreSQL REQUIRES every derived table to have an alias — forgetting it gives an immediate error; pick a short meaningful name.

Answer

✅ ANSWER

```
SELECT
  s.store_name,
  s.city,
  ROUND(rev.revenue, 0) AS total_revenue,
  rev.order_count
FROM stores.stores s
JOIN (
  SELECT
    store_id,
    SUM(net_total) AS revenue,
    COUNT(*) AS order_count
  FROM sales.orders
  GROUP BY store_id
) AS rev ON s.store_id = rev.store_id
ORDER BY total_revenue DESC
LIMIT 10;
```

Medium: Customers Above Their City Average

MEDIUM

SCENARIO: Marketing wants customers whose total spend EXCEEDS the average spend in their city. Need TWO derived tables: per-customer totals, and per-city averages. JOIN them and compare.

YOUR TASK: Derived table 1: customer_id, city, customer_total. Derived table 2: city, city_average. JOIN on city. Filter WHERE customer_total > city_average.

🤔 **Hint:** Two different aggregations at different grain levels (per-customer and per-city) — compute each in its own derived table, then JOIN on the shared dimension. Trap: trying to do both aggregates in ONE query forces multiple GROUP BY columns and breaks the per-city calculation; separation keeps each computation clean.

Answer (1/2)

✅ ANSWER

```
SELECT
  cust.customer_id,
  cust.customer_name,
  cust.city,
  ROUND(cust.cust_total, 0)
    AS customer_total,
  ROUND(city_avg.avg_spend, 0)
    AS city_average
FROM (
  SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name
      AS customer_name,
    a.city,
    SUM(o.net_total) AS cust_total
  FROM customers.customers c
  JOIN customers.addresses a
    ON c.customer_id = a.customer_id
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
  GROUP BY c.customer_id,
    c.first_name, c.last_name, a.city
```

Answer (2/2)

✅ ANSWER

```
) cust
JOIN (
  SELECT a.city,
    AVG(o.net_total) AS avg_spend
  FROM customers.addresses a
  JOIN sales.orders o
    ON a.customer_id = o.cust_id
  GROUP BY a.city
) city_avg ON cust.city = city_avg.city
WHERE cust.cust_total > city_avg.avg_spend
ORDER BY customer_total DESC LIMIT 15;
```

Hard: Multi-Source Dashboard (Pre-Aggregation Prevents Fan-Out)

HARD

SCENARIO: The Head of Operations needs a regional dashboard: per region, show store count, total employees, salary budget, total orders, and revenue. Multi-source aggregation — must avoid fan-out.

YOUR TASK: Derived table 1: per-store employee count + salary. Derived table 2: per-store orders + revenue. JOIN both to stores → dim_region. GROUP BY region.

🤔 **Hint:** Multi-source dashboards always suffer from fan-out when you JOIN raw tables directly — each side multiplies the other. Trap: COUNT(DISTINCT …) alone doesn't rescue SUMs from fan-out; only pre-aggregation per source keeps the math accurate.

Answer (1/2)

✅ ANSWER

```
SELECT
  r.region_name,
  COUNT(DISTINCT s.store_id) AS stores,
  COALESCE(SUM(emp.emp_count), 0)
    AS employees,
  COALESCE(SUM(emp.salary_budget), 0)
    AS salary_budget,
  COALESCE(SUM(ord.order_count), 0)
    AS orders,
  ROUND(COALESCE(SUM(ord.revenue), 0), 0)
    AS revenue
FROM core.dim_region r
JOIN stores.stores s
  ON r.region_id = s.region_id
LEFT JOIN (
  SELECT store_id,
    COUNT(*) AS emp_count,
    SUM(salary) AS salary_budget
  FROM stores.employees
  GROUP BY store_id
) emp ON s.store_id = emp.store_id
LEFT JOIN (
```

Answer (2/2)

✅ ANSWER

```
  SELECT store_id,
    COUNT(*) AS order_count,
    SUM(net_total) AS revenue
  FROM sales.orders
  GROUP BY store_id
) ord ON s.store_id = ord.store_id
GROUP BY r.region_name
ORDER BY revenue DESC;
```

Crazy: Manufacturing Defect Rate vs Line Average

CRAZY

SCENARIO: The Production Director wants production-line health: per work order, show rejected % AND how that compares to its production line's average rejection rate. Two derived tables: per-work-order rates and per-line averages. JOIN and compute the gap.

YOUR TASK: Derived table 1: per work_order, calculate rejection_pct (rejected / produced × 100). Derived table 2: per production line, average rejection_pct. JOIN. Show line, work_order, rate, line_avg, gap.

🤔 **Hint:** Two derived tables at different grain levels — one row per work order, one per line. JOIN on line_id. Trap: division by zero — a work order with zero produced returns NULL rejection_pct; wrap with NULLIF and the outer AVG silently drops those rows.

Answer (1/2)

✅ ANSWER

```
SELECT
  pl.line_name,
  wo_rates.work_order_id,
  wo_rates.rejection_pct,
  ROUND(line_avg.avg_pct, 2)
    AS line_avg_pct,
  ROUND(wo_rates.rejection_pct
    - line_avg.avg_pct, 2)
    AS gap_from_line
FROM (
  SELECT
    work_order_id, line_id,
    ROUND(100.0 * rejected_quantity
      / NULLIF(quantity_produced, 0), 2)
      AS rejection_pct
  FROM manufacture.work_orders
  WHERE quantity_produced > 0
) wo_rates
JOIN (
  SELECT
    line_id,
    AVG(100.0 * rejected_quantity
```

Answer (2/2)

✅ ANSWER

```
      / NULLIF(quantity_produced, 0))
      AS avg_pct
  FROM manufacture.work_orders
  WHERE quantity_produced > 0
  GROUP BY line_id
) line_avg
  ON wo_rates.line_id = line_avg.line_id
JOIN manufacture.production_lines pl
  ON wo_rates.line_id = pl.line_id
ORDER BY wo_rates.rejection_pct DESC
LIMIT 15;
```

KEY TAKEAWAYS

Derived Tables

Derived table = subquery in FROM treated as a table. Aliasing is REQUIRED. Use for pre-aggregation (compute group totals, then filter/join). Multiple derived tables in one query prevent fan-out in multi-table aggregations. Tomorrow's CTEs are the readable alternative — same idea, cleaner syntax.

PRO TIP

PRO TIP

Derived table vs CTE preview: both do the same thing — a CTE (WITH clause, tomorrow's topic) is just a NAMED derived table that you can reference multiple times. Use derived tables for one-shot inline pre-aggregation. CTEs win when complex logic needs to be readable or reused.

COMMON MISTAKE

Forgetting the Alias

The Mistake:
FROM (SELECT …) — no alias. PostgreSQL errors: 'subquery in FROM must have an alias'. Every derived table needs a name.

The Fix:
Always add AS alias: FROM (SELECT …) AS t. Use meaningful names: rev, agg, summary — not just t or x. Future you (debugging at 2 AM) will thank you.

STORY

Quick Fire Round!

Run each query. Try replacing the scalar value (50000) with a subquery. Modify the derived table grouping. Understanding comes from experimentation.

Challenge 1: Above-Average Orders

```sql
-- Orders larger than the average
SELECT order_id, net_total
FROM sales.orders
WHERE net_total > (
  SELECT AVG(net_total)
  FROM sales.orders
)
ORDER BY net_total DESC LIMIT 15;

-- YOUR TURN: Show the average too
-- as a column alongside net_total
```

Challenge 2: Products Bought in Mumbai

```
-- Products ordered by Mumbai customers
SELECT DISTINCT p.product_name, p.price
FROM products.products p
WHERE p.product_id IN (
  SELECT oi.prod_id
  FROM sales.order_items oi
  JOIN sales.orders o
    ON oi.order_id = o.order_id
  JOIN customers.addresses a
    ON o.cust_id = a.customer_id
  WHERE a.city = 'Mumbai'
)
LIMIT 15;

-- YOUR TURN: Rewrite as a JOIN
```

Challenge 3: Pre-Aggregation Pattern

```
-- Stores with revenue and order count
SELECT s.store_name, rev.revenue,
  rev.order_count
FROM stores.stores s
JOIN (
  SELECT store_id,
    SUM(net_total) AS revenue,
    COUNT(*) AS order_count
  FROM sales.orders
  GROUP BY store_id
) rev ON s.store_id = rev.store_id
ORDER BY rev.revenue DESC LIMIT 10;
```

Challenge 4: Scalar in SELECT

```
-- Every employee with company avg salary
SELECT first_name, salary,
  (SELECT ROUND(AVG(salary), 0)
   FROM stores.employees) AS company_avg,
  salary - (SELECT AVG(salary)
    FROM stores.employees)
    AS diff_from_avg
FROM stores.employees
ORDER BY diff_from_avg DESC LIMIT 15;
```

STORY

Answer Before Looking!

Test your subquery intuition before checking the answer.

Quiz 1: What Does This Return?

EASY

ANSWER: Names of employees earning above the company-wide average salary. The inner query returns ONE number (the average). The outer compares each employee's salary to it. Classic scalar subquery in WHERE.

Question

✅ ANSWER

```
SELECT name FROM stores.employees
WHERE salary > (
  SELECT AVG(salary)
  FROM stores.employees
);
```

Quiz 2: IN-Subquery vs JOIN

MEDIUM

Do these two queries return the same rows?

ANSWER: YES — same rows. Both find customers with at least one order. Performance varies: the optimizer often rewrites them to the same plan. Choose based on readability — IN-subquery is clearer when you only need rows from one table; JOIN+DISTINCT wins when you need columns from both.

Question

✅ ANSWER

```
-- Query A
SELECT * FROM customers c
WHERE c.customer_id IN (
  SELECT cust_id FROM sales.orders);

-- Query B
SELECT DISTINCT c.* FROM customers c
JOIN sales.orders o
  ON c.customer_id = o.cust_id;
```

Quiz 3: The NOT IN Trap

MEDIUM

ANSWER: Returns ZERO rows! When any value in the NOT IN list is NULL, the entire NOT IN evaluates to UNKNOWN → FALSE. Fix: WHERE prod_id IS NOT NULL in the inner query, or use NOT EXISTS (tomorrow).

Question

✅ ANSWER

```
SELECT * FROM products
WHERE product_id NOT IN (
  SELECT prod_id FROM order_items
);
-- What if order_items has NULL prod_ids?
```

Quiz 4: Scalar Subquery Limits

HARD

ANSWER: Will FAIL if dept 5 has more than one employee. Scalar subquery must return exactly ONE row, ONE column. Fix: aggregate — (SELECT MAX(salary) FROM employees WHERE dept_id = 5). Now it returns one number.

Question

✅ ANSWER

```
SELECT name,
  (SELECT salary FROM stores.employees
   WHERE dept_id = 5)
FROM core.dim_department;
-- What goes wrong?
```

Quiz 5: Derived Table Alias

CRAZY

What's wrong with this query?

ANSWER: Two problems. (1) Derived table has no alias — PostgreSQL requires one. (2) The aggregate column has no alias, so 'sum' (the function name) isn't directly addressable. Fix: AS total_rev outside the subquery and AS revenue inside the SUM.

Question

✅ ANSWER

```
SELECT * FROM (
  SELECT store_id, SUM(net_total)
  FROM sales.orders
  GROUP BY store_id
) WHERE sum > 100000;
```

COMMON MISTAKE

Scalar Position with Multiple Rows

The Mistake:
Using a multi-row subquery where SQL expects one value. PostgreSQL throws 'more than one row returned by a subquery used as an expression'.

The Fix:
Either aggregate to one row (MAX, MIN, AVG, SUM) OR change the operator from > to IN. The choice depends on intent — > MAX is 'greater than every value', IN is 'matches any value in the set'.

COMMON MISTAKE

NOT IN with NULL — Silent Failure

The Mistake:

NOT IN returns zero rows when the inner list contains NULL. No error, just wrong results. Easy to miss in code review.

The Fix:

Default to NOT EXISTS (tomorrow) for exclusion queries. It is safer with NULLs and often faster. If you must use NOT IN, ALWAYS add WHERE col IS NOT NULL in the inner query.

COMMON MISTAKE

Forgetting to Alias a Derived Table

The Mistake:

FROM (SELECT …) — no alias. PostgreSQL errors: 'subquery in FROM must have an alias'.

The Fix:

Always AS alias. Use meaningful names: rev, agg, t — not just x. Inside the subquery, also alias every aggregate column (SUM(x) AS total) so the outer query can reference them.

COMMON MISTAKE

Repeating Scalar Subqueries Many Times

The Mistake:

Pasting the same (SELECT AVG(...) FROM ...) into SELECT 5 times — the database may evaluate it 5 times unnecessarily.

The Fix:

Compute the value ONCE in a derived table or CTE, then reference the result column. Cleaner code, often faster execution. (CTEs arrive tomorrow.)

INTERVIEW

Scalar vs Multi-Row Subquery

Q: 'What's the difference between scalar and multi-row subqueries?'

A: Scalar returns ONE value (1 row, 1 col) — used in WHERE with =, >, <, and in SELECT as a computed column. Multi-row returns multiple rows — used with IN, NOT IN. Using a multi-row subquery in scalar position throws 'more than one row returned' error.

INTERVIEW

IN-Subquery vs JOIN Rewrite

Q: 'When would you rewrite an IN-subquery as a JOIN?'

A: When you also need columns from the inner table in the result. IN-subquery only returns columns from the outer table. JOIN gives you columns from BOTH. Performance-wise, the optimizer often rewrites them to the same plan; choose based on readability and the columns you need.

INTERVIEW

The NOT IN NULL Pitfall

Q: 'Why does NOT IN sometimes return zero rows when you expect matches?'

A: NULL handling. If the inner list contains ANY NULL, NOT IN evaluates to UNKNOWN (treated as FALSE) for ALL outer rows. Result: zero rows. Three-valued logic strikes again. Solutions: filter NULLs in inner query, OR switch to NOT EXISTS (tomorrow) which handles NULLs correctly.

INTERVIEW

Derived Table Best Practices

Q: 'When do you use a derived table in FROM?'

A: (1) Pre-aggregation before joining — prevents fan-out. (2) Filter or transform data before main logic. (3) Make a single computation reusable in multiple places. ALWAYS alias the derived table. Tomorrow's CTEs are the readable alternative for complex multi-step logic.

INTERVIEW

Subquery in SELECT — When?

Q: 'When would you put a subquery in the SELECT clause?'

A: For computed BENCHMARK columns — alongside each row, show a value computed from a different scope. Examples: each employee's salary AND the company average; each product's revenue AND the median revenue. Modern alternative: window functions (Database Concepts) — often faster and clearer for this pattern.

INTERVIEW

Scalar Subquery in WHERE — When?

Q: 'Give a real example of a scalar subquery in WHERE.'

A: 'Find products priced above the global average.' WHERE price > (SELECT AVG(price) FROM products). The benchmark (average) is DYNAMIC — computed from current data, not hardcoded. Re-runs always use today's numbers.

INTERVIEW

Multi-Row IN-Subquery Result

Q: 'How many rows can the subquery inside an IN clause return?'

A: ANY number — that's the point. Unlike scalar subqueries, IN happily handles 0, 1, 100, or 10,000 rows in the inner result. The outer row passes the filter if its value matches ANY of those rows. Zero rows in the inner result means the outer filter is always FALSE.

KEY TAKEAWAYS

KEY TAKEAWAYS

Scalar Subqueries:

1. Return ONE value. Use in WHERE, SELECT, HAVING.
2. Multiple scalar subqueries possible in one SELECT.
3. Error if subquery returns more than one row — aggregate or use IN.

IN / NOT IN Subqueries:

4. IN = match any value in the inner result set.
5. NOT IN has a NULL trap — always filter NULLs in inner query.
6. IN-subquery and JOIN+DISTINCT often give the same result.

Derived Tables:

7. Subquery in FROM, treated like a table. Alias REQUIRED.
8. Use for pre-aggregation — prevents fan-out in multi-source dashboards.
9. Multiple derived tables in one query handle multi-grain aggregates.
10. Tomorrow's CTEs are the readable alternative.

REAL WORLD

What You Achieved Today

You learned three subquery patterns that cover 80% of analyst needs. Scalar subqueries make WHERE conditions dynamic. IN-subqueries filter by membership in a computed set. Derived tables let you pre-aggregate before joining — the key to multi-source dashboards.

Combined with JOINs (earlier sessions), set ops (Subqueries Part 1), and aggregates (Aggregates & Grouping), you can now express almost any analyst question.

Tomorrow: Subqueries Part 2 — correlated subqueries (inner references outer), EXISTS (efficient existence check), and CTEs (named subqueries that read top-to-bottom). The senior-analyst level.

Subqueries Part 1

```
-- Scalar (WHERE)

 WHERE col > (SELECT AVG(x) FROM t)
 -- Inner returns ONE value
 -- Used in WHERE, SELECT, HAVING

-- Scalar (SELECT)

 SELECT a, (SELECT MAX(x) FROM t)
   AS max_x
 FROM table_a
 -- Benchmark column for each row

-- IN (membership)

 WHERE id IN (SELECT id FROM t)
 -- Inner can return many rows
 -- Matches if value appears in result
```

Subqueries Part 1

```
-- NOT IN (with NULL safety)
 WHERE id NOT IN (
   SELECT id FROM t
   WHERE id IS NOT NULL  -- safety!
 )

-- Derived Table (FROM)
 FROM (SELECT ... GROUP BY ...) AS t
 JOIN main ON main.id = t.id
 -- Alias REQUIRED

-- Pre-Aggregation Pattern
 LEFT JOIN (SELECT key, SUM(x) total
   FROM detail GROUP BY key) sub
 ON main.key = sub.key
 -- Prevents fan-out
```

CHALLENGE

Subqueries Part 2 Take-Home Challenge (1/2)

Build these queries on RetailMart V3:

Easy (Scalar Subqueries):

1.: Orders larger than the average order value. Show order_id, net_total, and the average.

2.: Products priced below the global average product price. Show product, price, and avg.

CHALLENGE

Subqueries Part 2 Take-Home Challenge (2/2)

Medium (IN / Subquery in SELECT):

3.: Customers who ordered Electronics products. Use IN-subquery through dim_brand/dim_category.

4.: Each customer with their lifetime order count AND this-month order count (subquery in SELECT).

Hard (Derived Tables):

5.: Top 5 stores by revenue using pre-aggregation (derived table).

6.: Customers above their city's average spend (two derived tables, JOIN on city).

Crazy:

7.: Rewrite ONE IN-subquery as both: (a) JOIN+DISTINCT, (b) EXISTS (preview tomorrow). Compare row counts and reasoning.

Push your SQL file to GitHub!
https://github.com/starlordali4444/PostgreSql

STORY

Coming Tomorrow

Correlated subqueries — inner query references the OUTER row, runs per-row. EXISTS / NOT EXISTS — fast existence checks, NULL-safe alternatives to IN. CTEs (WITH clause) — named subqueries that read top-to-bottom. Recursive CTEs for org charts and category trees.

CTEs is where SQL composition really kicks in. After tomorrow, you'll write queries that look more like programs than database commands.